Tree predictors for binary classification

Project 2

Alessandro Di Gioacchino

January 2025

1 Overview

The binary classification task is a fundamental problem in machine learning, where the goal is to predict which of two classes a given data point belongs to. This project focuses on implementing tree predictors for binary classification to determine whether mushrooms are poisonous. The Mushroom dataset [WHH21], obtained from the UCI Machine Learning Repository, is used for this purpose.

Tree predictors are a popular and effective method for binary classification because they are relatively easy to train and interpret. The project's main task is to implement tree predictors from scratch using single-feature binary tests as the decision criteria at any internal node. We consider thresholds on a single feature in the case of numerical/ordinal attributes or membership tests in the case of categorical attributes.

The project involves implementing a node class/structure and a tree predictor class/structure, defining the decision criteria/tests, splitting and stopping criteria, and training and evaluation procedures. We train the tree predictors using at least three reasonable criteria for expanding the leaves and at least two reasonable stopping criteria. We then evaluate the performance of each tree predictor using the 0-1 loss and perform hyperparameter tuning to optimize their performance.

The report is organized as follows. In Section 2, we describe the dataset and how it was preprocessed. In Section 3, we present the methodology used in this project, including the structure and attributes/procedures of the node and tree predictor classes, the decision criteria/tests, splitting and stopping criteria, and the training and evaluation procedures; we also provide some implementation details of the node and tree predictor classes. In Section 4, we discuss the hyperparameter tuning procedure used and the behavior displayed by the model for varying values. Finally, in Section 5 we show the results obtained, including the training error of each tree predictor, and the performance of each tree predictor with different criteria for expanding leaves and stopping criteria.

2 Dataset

The dataset contains around 60 thousand rows of (simulated) mushrooms, and 20 different features for each. Some of these, namely `stem-root`, `stem-surface`, `veil-type`, `veil-color`, `spore-print-color`, are dropped because the majority of their values is `NaN`. Then we delete those examples still containing `NaN`s in any of the features, resulting in a total of 20 thousand instances more or less. From this point of view, a possible improvement would be to properly handle missing values, instead of just discarding entire examples, but 20 thousand examples are more than enough to build a tree; also, the more examples we have in the train set the more time training requires and, as we shall see, this is already an issue with "small" datasets. So the rationale behind removing these examples is, on the one hand, that we need a smaller dataset for performance reasons, and we might as well remove incomplete rows instead of extracting some random sample; on the other hand, the current implementation doesn't mix well with `NaN`s: in particular, the function `np.unique`, provided by Numpy, is needed to extract the unique values of each attribute, and it causes errors when `NaN`s are present (no optional parameter seems to fix this behavior).
The remaining features follow [Mus] [WHH].

- Features about the top portion of mushrooms (pileus):
  - `cap-diameter`, continuous
  - `cap-shape`, categorical
  - `cap-surface`, categorical: contains a property of the cap surface
  - `cap-color`, categorical
  - `does-bruise-or-bleed`, categorical: "bruising" is the process of scratching the top and bottom of a mushroom cap, and observing subsequent color changes (if any); "bleeding" occurs after cutting some species of mushrooms, and it's exactly what it sounds like (except, of course, no blood comes out but a white substance)
- Features about the underside of the cap of some mushrooms, where spores are stored (lamellae):
  - `gill-attachment`, categorical: the way gills are connected to the stem
  - `gill-spacing`, categorical: whether gills are tightly packed or more widely apart
  - `gill-color`, categorical
- Features about the cylinder supporting the cap and the spores it contains:
  - `stem-height`, continuous
  - `stem-width`, continuous
  - ~~`stem-root`, categorical: contains a property of the stem root~~ (discarded feature)
  - ~~`stem-surface`, categorical: contains a property of the stem surface, like `cap-surface`~~ (discarded feature)
  - `stem-color`, categorical
- Features of the veil, a layer protecting the gills of some mushrooms in the early stages of growth:
  - ~~`veil-type`, categorical~~ (discarded feature)
  - ~~`veil-color`, categorical~~ (discarded feature)
- Features about the ring, formed after the veil breaks away from the cap and is left hanging around the stem:
  - `has-ring`, categorical
  - `ring-type`, categorical: contains a property of the ring
- ~~`spore-print-color`, categorical: the color of spores fallen to a surface underneath~~ (discarded feature)
- `habitat`, categorical
- `season`, categorical

As we can see, we only have three continuous attributes, so it makes sense for the tree we are implementing to handle them specifically. Scikit-learn's `DecisionTreeClassifier` [scia], possibly the most widely known implementation of tree predictors, requires instead to encode categorical features somehow, as it only works on numerical attributes.
The target of this dataset is represented by the `class` attribute: mushrooms are either edible or poisonous and, given a new mushroom we have never seen, we want the trained tree to tell us whether it is edible. The poisonous class also contains mushrooms of unknown edibility [WHH], so it would be interesting to see what a tree predictor says about them; unfortunately, they are not marked in any way, so we don't know exactly what mushrooms are poisonous and what may be edible.
Classes are more or less balanced: after removing rows with `NaN` values, we are left with 12355 poisonous and 9884 edible mushrooms. A tree predictor always returning the poisonous label would do slightly better than a random predictor (this information could be useful for hyperparameter tuning).
The plot shown in figure 1 was obtained by extracting the first two principal components, and then sampling the dataset; it tells us that classifying the examples is not trivial, although the dataset could still be linearly separable in the original space. To make sure, we also trained sklearn's perceptron [scib] on the dataset, which should find

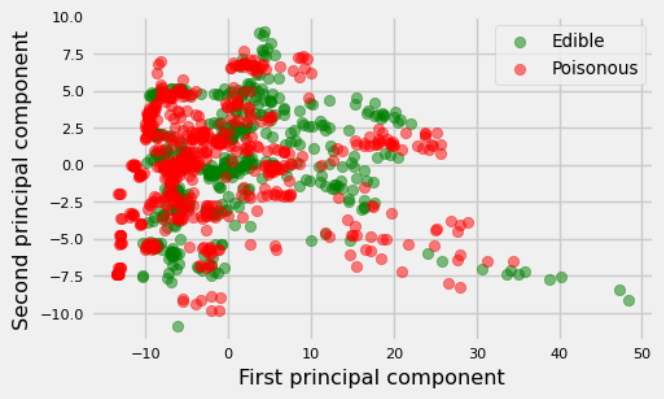

Figure 1: Scatter plot obtained by applying PCA to the dataset, and plotting the first two principal components.

the separating hyperplane if there is one. The resulting score is around 0.45, meaning that the dataset is unlikely to be linearly separable.

3 Methodology

3.1 Nodes

The node class has a simple structure. There is a constructor that initializes the node and its attributes. The attributes include a decision criterion, that takes a data point as input and returns a Boolean value as output, left and right children, a label. Decision criterion and left and right children are relevant only for internal nodes, where the decision criterion is used to route an example to one of the children. The label is only relevant for leaf nodes, and it represents the prediction (poisonous or edible) for an example that reaches that leaf.
The class has only one method, which checks whether a node is a leaf. While this information could have been stored as an attribute, using a method without changing the class's state makes it easier to maintain the consistency of the objects. Specifically, an internal node will never mistakenly identify itself as a leaf.

3.2 Trees

The tree predictor class has several important attributes and methods. A list of categorical features is required to instantiate a tree predictor, as they need treatment different from continuous features. Among the optional arguments are the criterion used to split leaves and the hyperparameters to stop tree growth prematurely.
The tree predictor class includes the classic `fit` and `predict` methods, used to build the tree for a given dataset and predict the labels for the provided examples. The method to grow the tree selects an attribute and a test for the current tree and, based on the stopping criteria, determines if a leaf can be split to create a new internal node, with left and right children. The procedure to select an attribute and a test is the most expensive operation of the class.
For a continuous feature, the approach is to consider all unique values as thresholds for the test. When an example arrives at the internal node, if the attribute the test selects is smaller than or equal to the threshold, the example is routed to the left child; otherwise, it is routed to the right one. For a categorical feature, we consider the power set of its unique values as membership tests. An example would then be routed to the left child if its selected attribute is in the chosen subset, and to the right one otherwise. Among all possible tests, we choose the one that maximizes the information gain obtained with the split, which is the difference between the splitting criterion computed on the internal node and the criterion computed on its children.
The few hyperparameters that we have considered for the tree predictor are

- `max_depth`: constraint on the height of the tree;
- `min_samples_split`: constraint on the minimum number of samples routed to a leaf for a split to be explored;
- `min_impurity_decrease`: constraint on the minimum amount of information gain for a split.

An additional hyperparameter is slightly different from the others because it depends on the model implementation. The name is `n_unique_values`, and it controls the number of unique values considered to compute the information gain before a split. This hyperparameter was only introduced for performance reasons: there is no reason to discard some unique values other than that.

Implementation details

The implementation of the class for tree predictors is not overly complex. One notable aspect is the usage of a separate class implementing splitting criteria: objects of this class are passed to `__init__` function in place of the usual string (e.g. `'gini'` or `'entropy'`). The main advantages of this approach are that no (static) method is implemented in the tree class to compute the heterogeneity index of a given split; also, there are no `if`s on the parameter representing the splitting criterion: we invoke the same method on the object for all splitting criteria, and that's it.
Another interesting point is the use of closures in the function to build the tree. Without going into the details, a closure happens when a function takes its parameters from an outer function. In our case, we always use the same decision criterion inside nodes (although its structure changes depending on the type of feature it works on): the difference is in its parameter, which the criterion takes from the function building the tree (remember this loops over all unique values of an attribute).
Performance-wise, this custom tree cannot compete with e.g. scikit-learn's: tests have shown that the difference is a couple of orders of magnitude. Regarding learning, we are talking about seconds for sklearn's, and at least

hundreds of seconds for the custom. The bottleneck is most likely caused by the loop over all possible unique values for each attribute, and recursion could also play a part in the complexity. A small workaround implemented is to sample the unique values, instead of considering each of them, but even so the custom tree is much slower than sklearn's (not to mention that we are effectively discarding precious information).

Splitting criteria

As previously stated, splitting criteria are not simple strings but full-fledged objects. This was implemented through a small hierarchy, at the top of which we have an abstract class representing any splitting criterion, and at the second level we have some concrete classes.
The abstract class requires implementing method φ, which computes the heterogeneity index on a vector of binary labels. Then, it leverages this method to implement a "default" one that calculates the information gain of a given split.
Concrete classes only need to implement φ. The ones already provided are:

- Gini index, $2p\,(1-p)$
- scaled entropy, $-\frac{p}{2}\,\log_2(p) - \frac{(1-p)}{2}\,\log_2(1-p)$
- standard deviation of the Bernoulli random variable, $\sqrt{p\,(1-p)}$

All of them are symmetric functions with a maximum of $1/2$ achieved on $p = 1/2$.

Correctness

Three different approaches have been adopted to determine whether the custom tree is behaving correctly, all consisting of moving labels around.
First of all, we separate the train and test sets using sklearn's `train_test_split`. Then, we focus on the train labels `y_train`: what would happen if we swap them uniformly at random, without moving the corresponding example? For starters, a tree trained on these labels should differ from a tree trained on the original labels. Since this is a binary classification problem, we expect that around 50% of the examples have retained the original label, while the other half now has a different one. If the implementation is correct, the tree we obtain after training should behave randomly on the test set.
That is exactly what happens: the zero-one loss on the test set is around 0.47; the 0.03 point we are missing is probably related to the fact that the dataset is not perfectly balanced. To be safe, we can also compute the same measure on the train set: this is 0.18, a number that doesn't tell us much. We need first to consider what the training error of a tree with the same hyperparameters (only the value for `max_depth` is changed from the default) is on the original dataset: it is zero. This tells us that swapping train labels made the train set a bit harder to learn, and a higher depth is required to achieve a zero training error.
To check that our hypothesis is not completely arbitrary, we can try the same thing using sklearn's `DecisionTreeClassifier` instead. And sure enough, the zero-one loss on the test set is approximately the same (0.47); the zero-one loss on the train set is larger than our custom tree, as it too is 0.47.
Next, we can try something similar, but we move test labels around this time. This may be easier to imagine: the tree learns something from the original train set, but it isn't useful to make predictions on the test set because around half of the labels are now different. Once again, the resulting tree behaves almost randomly on the test set: it achieves a zero-one loss of approximately 0.48 (and a zero training error). This result also shows that the tree isn't peeking at the test set while training.

4 Hyperparameter tuning

As we have already seen, all hyperparameters of our tree predictor are constraints on some properties of the tree itself: the process of growing is prematurely stopped if any of these constraints are met. They could be useful to prevent overfitting, but our tests on unconstrained trees showed no such behavior, at least on the Mushroom dataset. This means that an unconstrained tree predictor trivially achieves the best performance.
We still defined a function to perform hyperparameter tuning, mainly for academic purposes. Without entering into implementation details, the function splits the dataset into three folds: two are used as the outer train set, and one as the test set. The train set is split again into three other folds: two are used as the inner train set, and one is used as the validation set. Any of the three folds can be used as validation set (and the remaining two as train set), so for each of these three combinations we train a tree with the desired hyperparameter values on the train set, and test it on the validation set. Finally, we take the hyperparameters associated with the best model, build a new tree on the outer train set, and evaluate its performance on the test set.
Time-wise, such an operation is costly. As previously stated, the bottleneck is mainly given by checking the set of unique values for splits. For this reason, an even smaller sample than what we already considered was drawn from these values. Since we are testing fewer values, hyperparameter tuning is faster, but we still had to choose a tiny set of values for the hyperparameters. Specifically, we only tested the following:

- `max_depth`: $5, 10, 15$
- `min_samples_split`: $50, 500, 1000$
- `min_impurity_decrease`: $0.02, 0.05, 0.07$

Despite the limited number of values, the procedure required around two hours. Unsurprisingly, the result tells us what we already guessed: smaller values of `min_samples_split` and `min_impurity_decrease` lead to better accuracy, because the selected values for each of the resulting trees are, respectively, 50 and 0.02. On the other hand, no tree grew further than 10 levels, possibly meaning that overfitting starts to show with a depth of 15. All trees achieve an accuracy of around 97%.
We can also see how hyperparameters affect the accuracy of the model: we expect a decreasing accuracy for increasing values of `min_samples_split` and `min_impurity_decrease`.
The plot in figure 2 shows that even with large values of the `min_samples_split` hyperparameter, the model retains acceptable levels of accuracy.

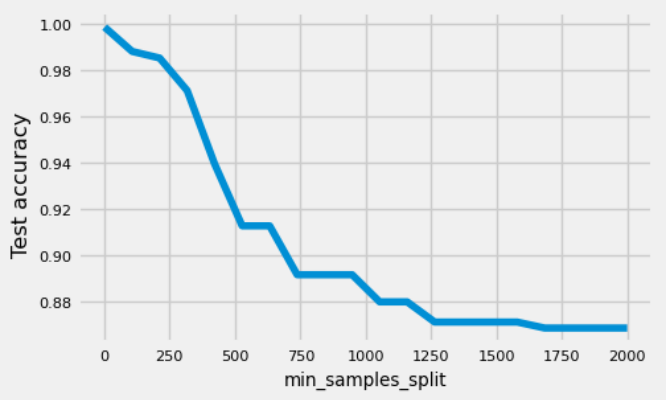

Figure 2: Plot representing test accuracy against 20 different values of the `min_samples_split` hyperparameter in the interval $[2, 2000]$.

`min_impurity_decrease` is more unforgiving instead: if we set it to a value between 0.04 and 0.05, the accuracy falls below 60%. What is likely happening here is that the tree doesn't grow beyond the root, and the prediction is simply the majority label of the dataset (more precisely, of the train set). We can see this behavior in 3.
The hyperparameters affect the time to train similarly: smaller values require more time, and larger values require less. Of course, as we can see in figure (4), decreasing values of `min_samples_split` approach the minimum time more slowly.
While, in figure 5, `min_impurity_decrease` goes quickly down to the minimum time, of course in correspondence to the point between 0.04 and 0.05 where the test accuracy collapses.
The time to train becomes an increasing function for the other two hyperparameters. Figure 6 shows how `max_depth` affects the time to train, figure 7 does the same for `n_unique_values`.

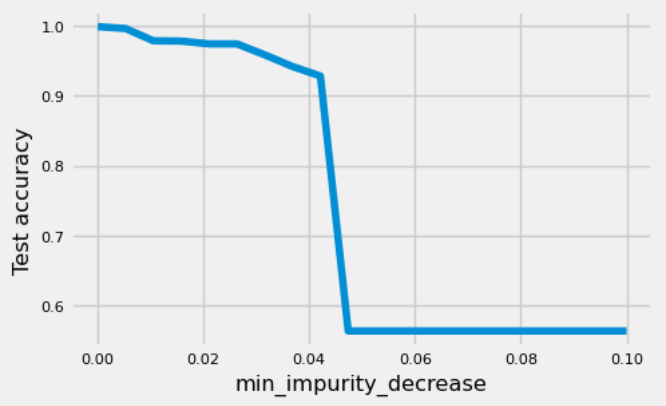

Figure 3: Plot representing test accuracy against 20 different values of the `min_impurity_decrease` hyperparameter in the interval $[0.0, 0.1]$.

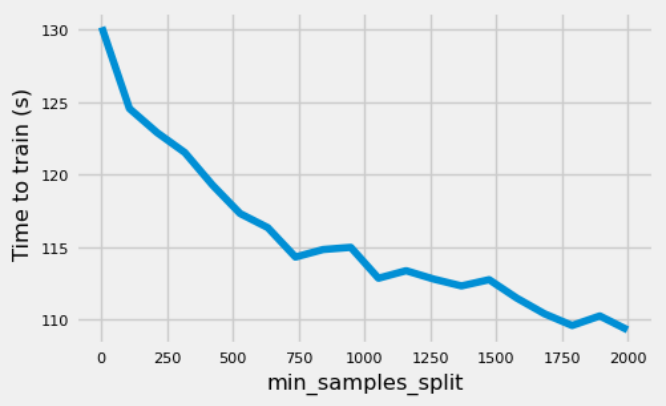

Figure 4: Plot representing the time to train (in seconds) against 20 different values of the `min_samples_split` hyperparameter in the interval $[0, 2000]$.

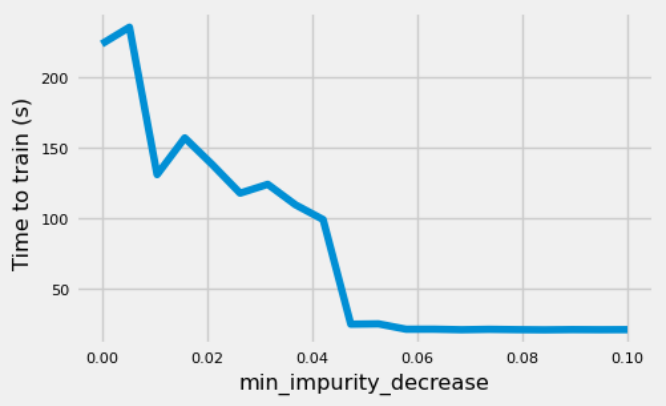

Figure 5: Plot representing the time to train (in seconds) against 20 different values of the `min_impurity_decrease` hyperparameter in the interval $[0.0, 0.1]$.

It's important to note that all plots concerning time are highly susceptible to the machine's load where code is executed. They should be monotonic functions, but we can see some spikes here and there: this is likely due to external factors, unrelated to the dataset or the model.
Finally, figure 8 displays how the number of unique values considered for splits influences the test accuracy: intuitively, this function should be increasing but, while the maximum is reached with larger values of `n_unique_values`, the rest of the function behaves erratically, oscillating between large and (relatively) small values of accuracy. The only explanation I could think of is that, since a sample is extracted uniformly at random from the entire set of unique values, some samples are luckier than others and result in better splits.

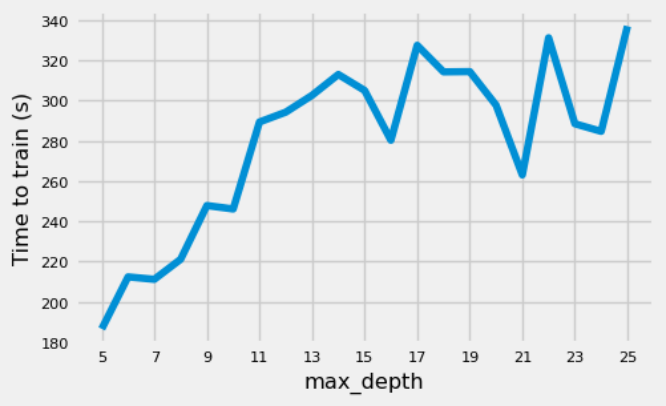

Figure 6: Plot representing the time to train (in seconds) against 21 different values of the `max_depth` hyperparameter in the interval $[5, 25]$.

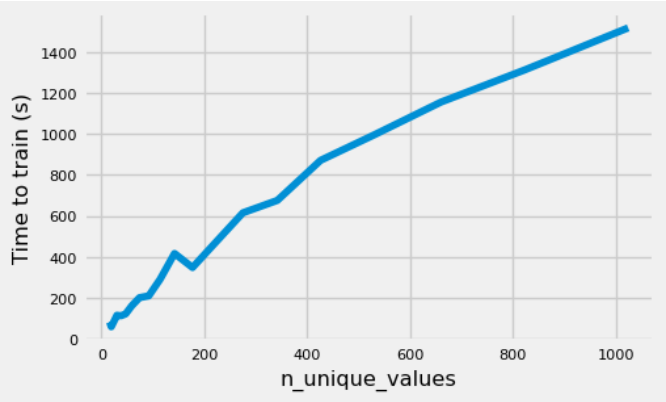

Figure 7: Plot representing the time to train (in seconds) against 20 different values of the `n_unique_values` hyperparameter in the interval $[2^4, 2^{10}]$.

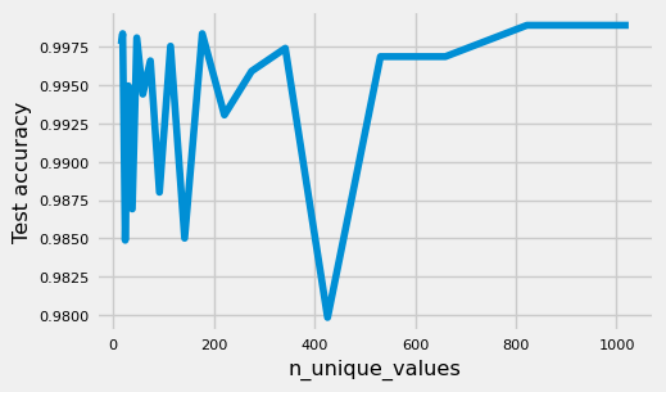

Figure 8: Plot representing the test accuracy against 20 different values of the `n_unique_values` hyperparameter in the interval $[2^4, 2^{10}]$.

Then, we have a comparison between train and test errors in figure 9: contrary to our previous assumption, larger values for `max_depth` do not result in overfitting, as test and train accuracy go hand in hand. It is also possible that the dataset isn't "hard" enough to require unbounded trees, so it could be that no tree reaches a depth of 25. Whatever the case, `max_depth` values greater than 12 are not especially useful for the Mushroom dataset.

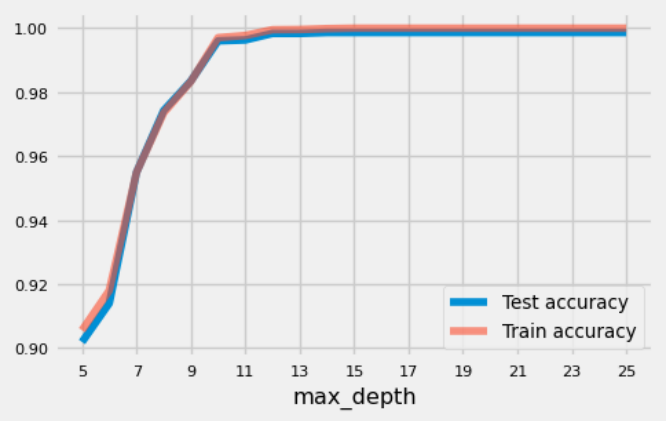

Figure 9: Plot representing test and train accuracy against 21 different values of the `max_depth` hyperparameter in the interval $[5, 25]$.

5 Results

Overall, a tree predictor is capable of learning and generalizing pretty well for what concerns the Mushroom dataset.
When using the Gini index as a splitting criterion and constraining the minimum impurity decrease at 0.07, meaning splits with an information gain less than or equal to 0.07 are ignored, the zero-one loss on the test set is 0.12; of the 20 considered splits, six have been discarded because their information gain is too small. The corresponding tree predictor is shown in figure 10, while its confusion matrix is displayed in table 1.

		Prediction		
		Edible	Poisonous	Total
Label	Edible	2613	269	2882
	Poisonous	591	3866	4457
	Total	3204	4135	7339

Table 1: Confusion matrix for the tree predictor using the Gini index as splitting criterion.

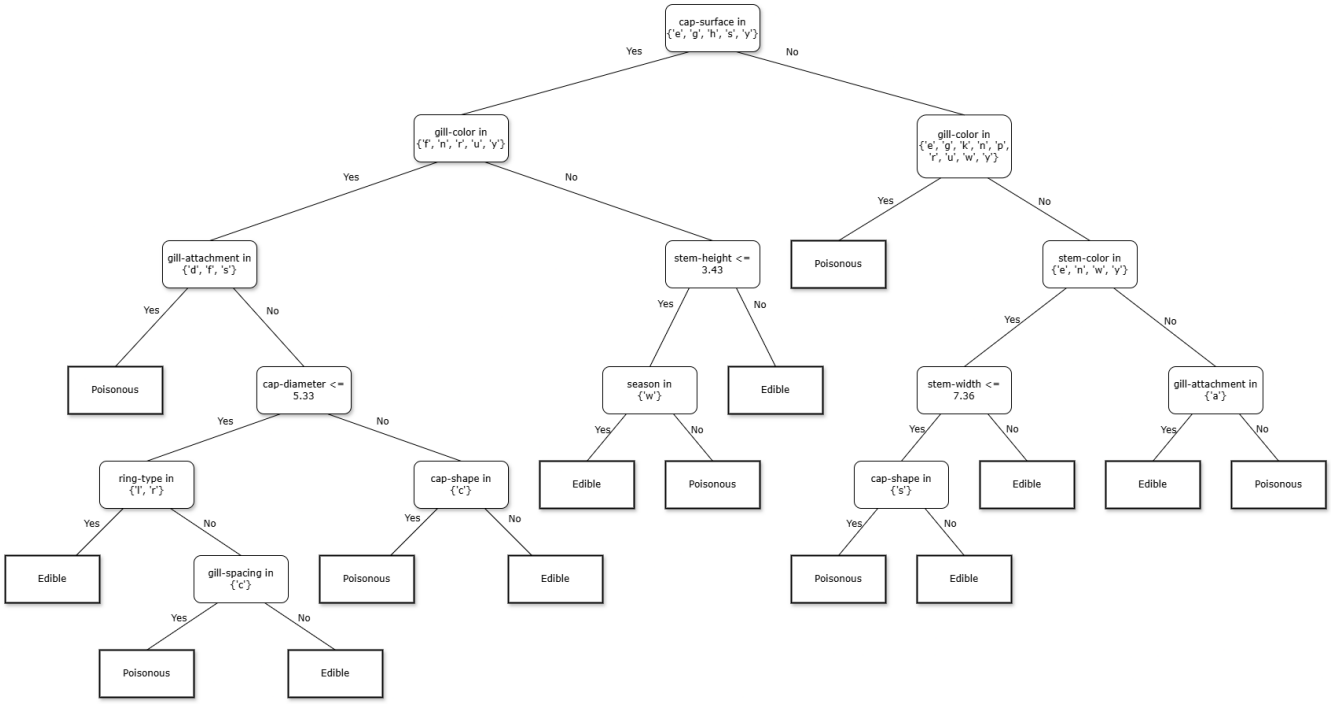

Figure 10: Tree predictor built using the Gini index as splitting criterion, and a minimum impurity decrease of 0.07 as stopping criterion.

When using the scaled entropy as a splitting criterion and constraining at 4096 the minimum number of samples to split, the zero-one loss on the test error is 0.16. Eight splits out of 16 were ignored because of the constraint. The resulting tree predictor is shown in figure 11, and the confusion matrix in table 2.

		Prediction		
		Edible	Poisonous	Total
Label	Edible	2379	380	2759
	Poisonous	825	3755	4480
	Total	3204	4135	7339

Table 2: Confusion matrix for the tree predictor using scaled entropy as splitting criterion.

When using the standard deviation and constraining at five the maximum tree depth, the zero-one loss on the test error is 0.14; four out of 14 splits are suppressed because of the depth constraint. The tree is shown in figure 12, with the corresponding confusion matrix in table 3.

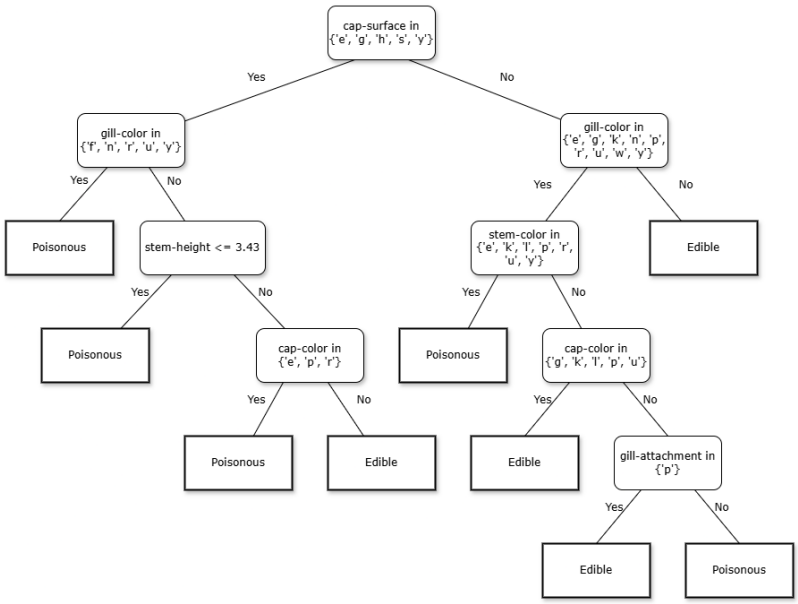

Figure 11: Tree predictor built using scaled entropy as splitting criterion, and a minimum sample number of 4096 as stopping criterion.

		Prediction		
		Edible	Poisonous	Total
Label	Edible	2922	774	3696
	Poisonous	282	3361	3643
	Total	3204	4135	7339

Table 3: Confusion matrix for the tree predictor using standard deviation as splitting criterion.

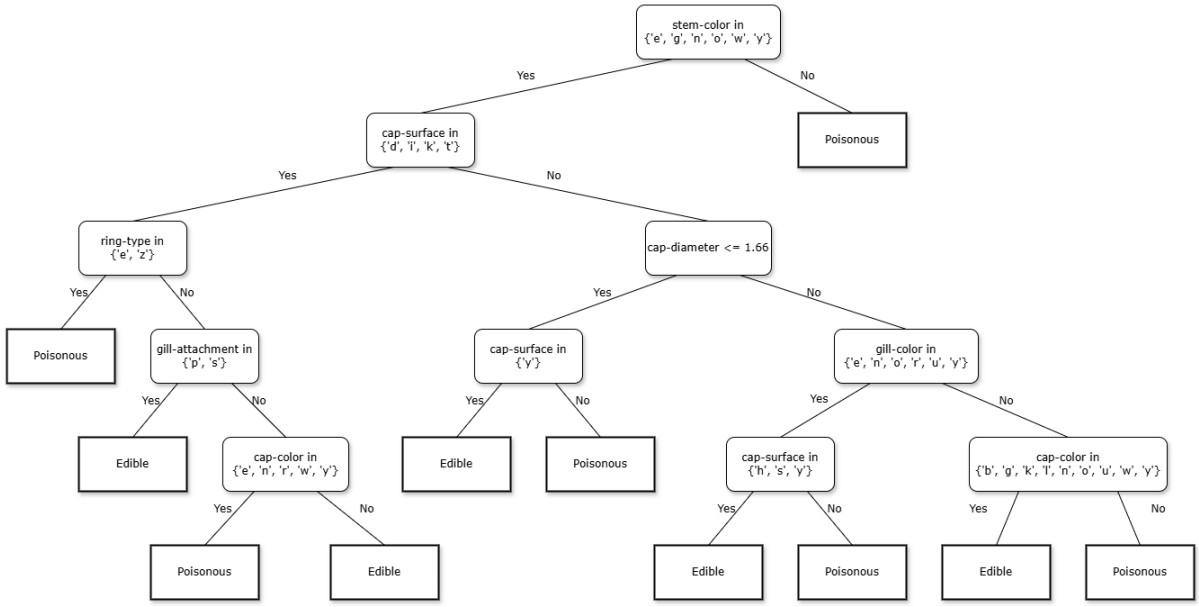

Figure 12: Tree predictor built using the standard deviation as splitting criterion, and a maximum depth of five as stopping criterion.

References

[WHH21] Dennis Wagner, Dominik Heider, and Georges Hattab. "Mushroom data creation, curation, and simulation to support classification tasks". In: *Scientific Reports* 11.1 (Apr. 2021). ISSN: 2045-2322. DOI: 10.1038/s41598-021-87602-3. URL: http://dx.doi.org/10.1038/s41598-021-87602-3.

[Mus] Real Mushroom. *Mushroom Anatomy: A Deep Dive Into the Parts of a Mushroom*. URL: https://www.realmushrooms.com/mushroom-anatomy-parts/. (accessed: 21.01.2025).

[scia] scikit-learn. *DecisionTreeClassifier*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html. (accessed: 21.01.2025).

[scib] scikit-learn. *Perceptron*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html. (accessed: 21.01.2025).

[WHH] Dennis Wagner, D. Heider, and Georges Hattab. *Secondary Mushroom*. URL: https://archive.ics.uci.edu/dataset/848/secondary+mushroom+dataset. (accessed: 21.01.2025).